VYBORA

E-Commerce Website Hosting on AWS S3

Project documentation

A screenshot-supported implementation record for a static storefront hosted using Amazon S3 website hosting.

PROJECT SCOPE Static front-end website deployment and hosting demonstration. The supplied evidence does not establish a production e-commerce backend, payment processing, domain configuration, HTTPS delivery, or CloudFront distribution.

COMPILED BY TEJAS SUDHAKAR KAMBLE

At a glance

Project	VYBORA — E-Commerce Website Hosting on AWS S3
Application artifact	Website-Landing-Page/index.html
Hosting service evidenced	Amazon S3 static website hosting
Evidence reviewed	16 numbered screenshots plus supplied HTML source
Primary outcome	Storefront content is shown loading from an S3 website endpoint in a browser

1. Project overview and objectives

The project pairs a branded retail landing page with an AWS-hosted delivery path. Its central objective is to make the supplied static storefront available through Amazon S3 website hosting and to document the process in a reproducible, evidence-based format.

Objectives

- Prepare a polished storefront experience for product discovery across Tech, Style, Lifestyle, and Home categories.
- Deploy the provided HTML entry point to an S3 bucket configured for static website hosting.
- Capture the setup, upload, and browser-rendered result with numbered screenshots.
- Explain the distinction between static demo interactions and production commerce capabilities.

Storefront capabilities visible in source

The page includes navigation anchors, category tiles, an eight-item in-page product dataset, category filters, text search, a demo cart count, favourite toggles, responsive CSS breakpoints, and reduced-motion styling. Product prices and ratings are present as front-end display content; the source itself labels ratings as demo ratings. These interactions run in the browser and do not imply persistent orders or server-side inventory.

2. Technologies and project assets

- Amazon S3 — bucket storage and static website hosting, as evidenced by the AWS console screenshots.
- HTML5 — semantic page structure and the bundled content.
- CSS — embedded styles for visual design, responsive layouts, and reduced-motion preferences.
- Vanilla JavaScript — in-browser rendering, filtering, search, cart counter, toast messages, and favourite toggles.
- Web browser — used in the screenshots to access the hosted site.

The archive contains one website source file (`index.html`) and 16 PNG screenshots. The reviewed HTML contains no references to AWS SDKs, external API calls, a database, a payment gateway, or a server-side application. No separate build tooling or deployment automation is included in the archive.

3. AWS architecture and workflow

The demonstrated path is intentionally simple: a browser requests the static website endpoint provided by the S3 website-hosting configuration; S3 returns the uploaded website object. The

PROJECT BY TEJAS SUDHAKAR KAMBLE

deployment evidence is consistent with a publicly readable static website. There is no evidence of an application server or API tier.

Component	Role in the demonstrated setup
Browser	Requests and renders the storefront from the S3 website endpoint.
Amazon S3 bucket	Stores the static site object and provides website hosting configuration.
index.html	Single delivered entry file containing HTML, embedded CSS, and JavaScript.

BOUNDARY Screenshots show bucket setup and an S3 website endpoint. They do not verify a custom domain, TLS/HTTPS via CloudFront, DNS, cache policy, monitoring, automated deployment, or a backend.

4. Implementation summary (screenshots in numerical order)

The figure sequence below follows the archive's numbered screenshots. Captions describe only what is visible; configuration choices not legible or not shown are not inferred.

4.1 Locate S3 and create the bucket

Screenshot 1 shows the AWS console service search selecting S3. Screenshot 2 shows the S3 landing page and the Create bucket action. Screenshots 3–6 document the bucket-creation form: region/general configuration, object ownership (with ACLs enabled), Block Public Access controls, and default encryption. Screenshot 6 specifically shows SSE-S3 (S3-managed keys) selected and the S3 Bucket Key enabled. The screenshots show public-access controls being reviewed/changed in the setup flow; the final effective permissions are not audited here.

PROJECT BY TEJAS SUDHAKAR

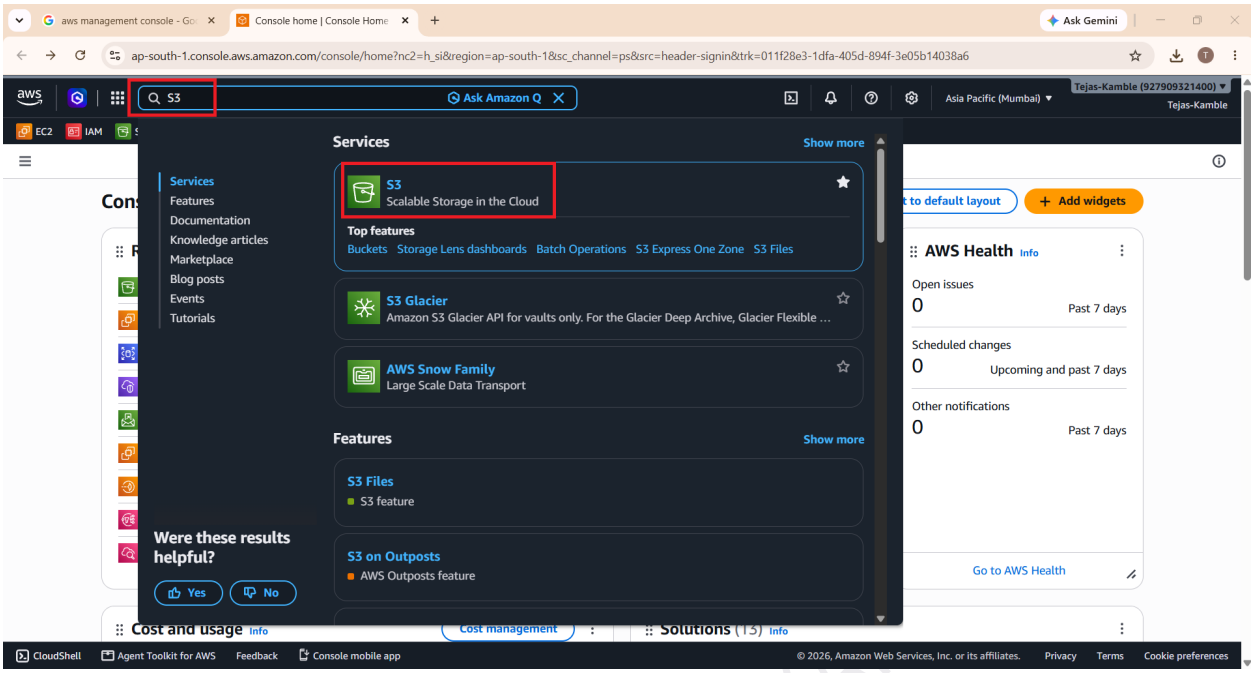


Figure 1. AWS console service search with S3 selected.

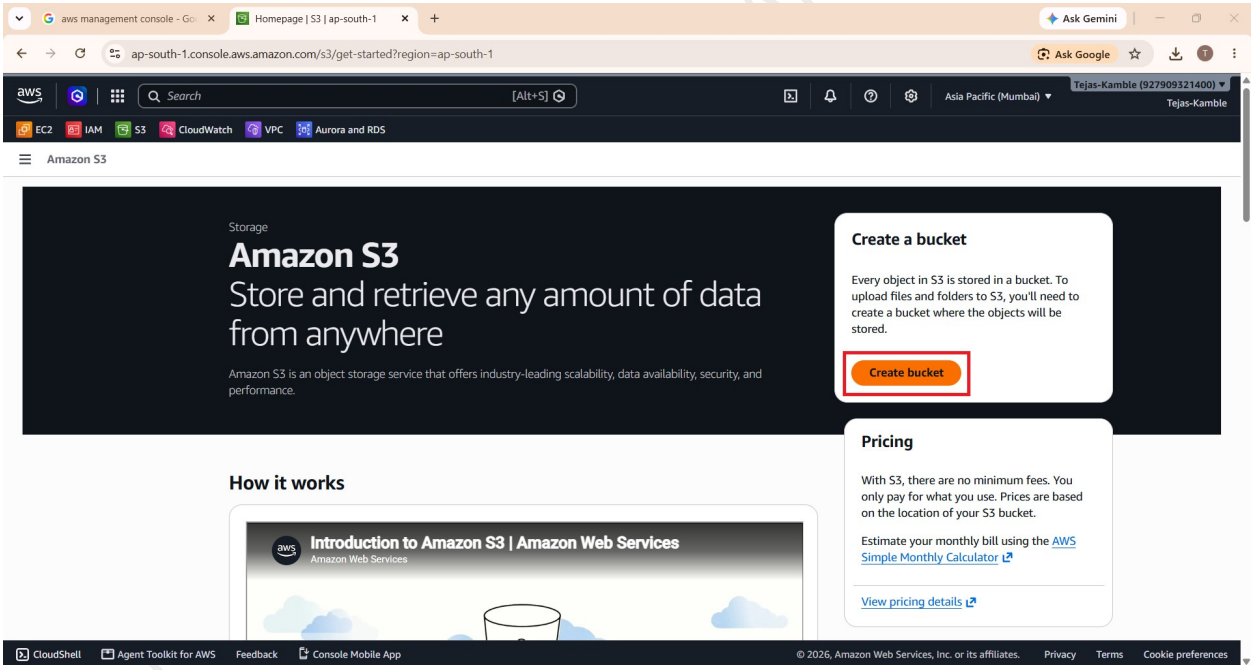


Figure 2. Amazon S3 console and bucket-creation entry point.

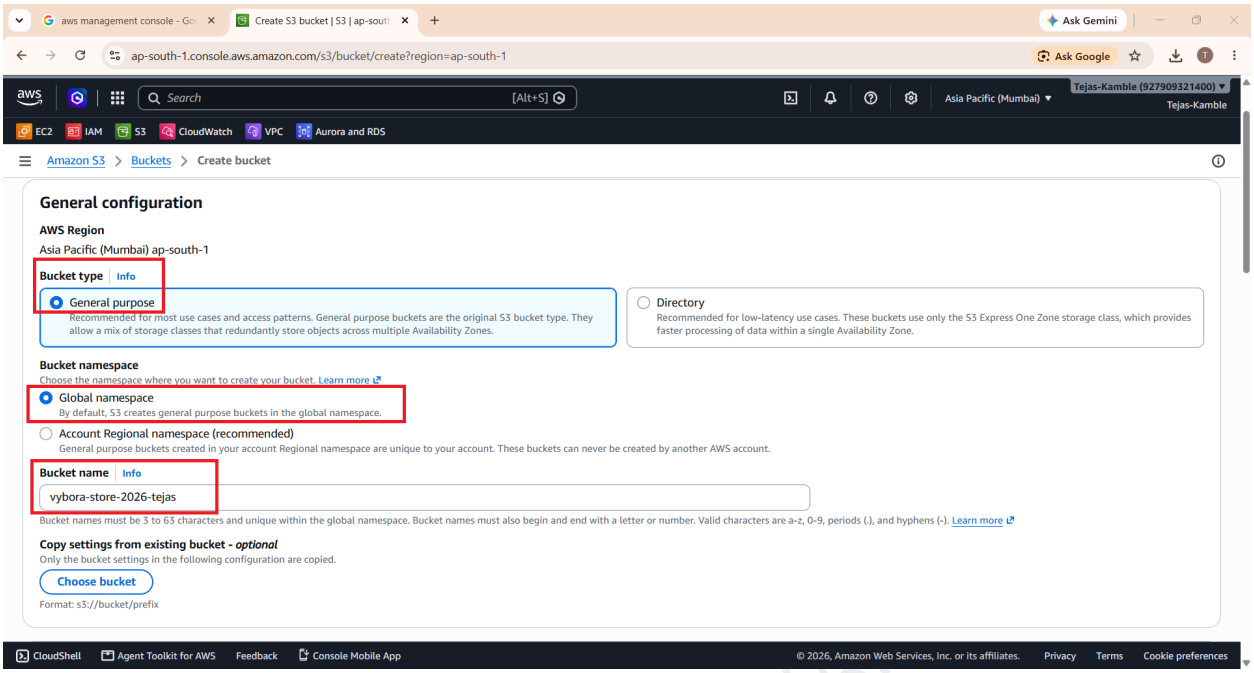


Figure 3. Bucket creation form: general configuration.

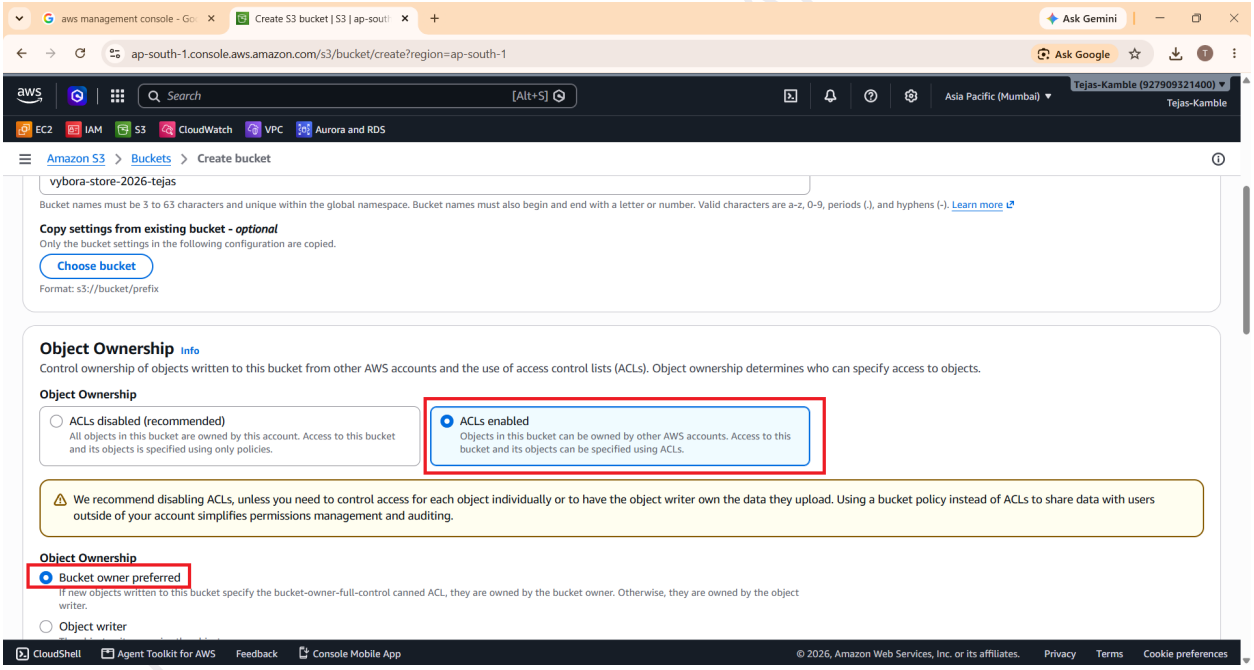


Figure 4. Bucket creation form: ownership and public-access settings.

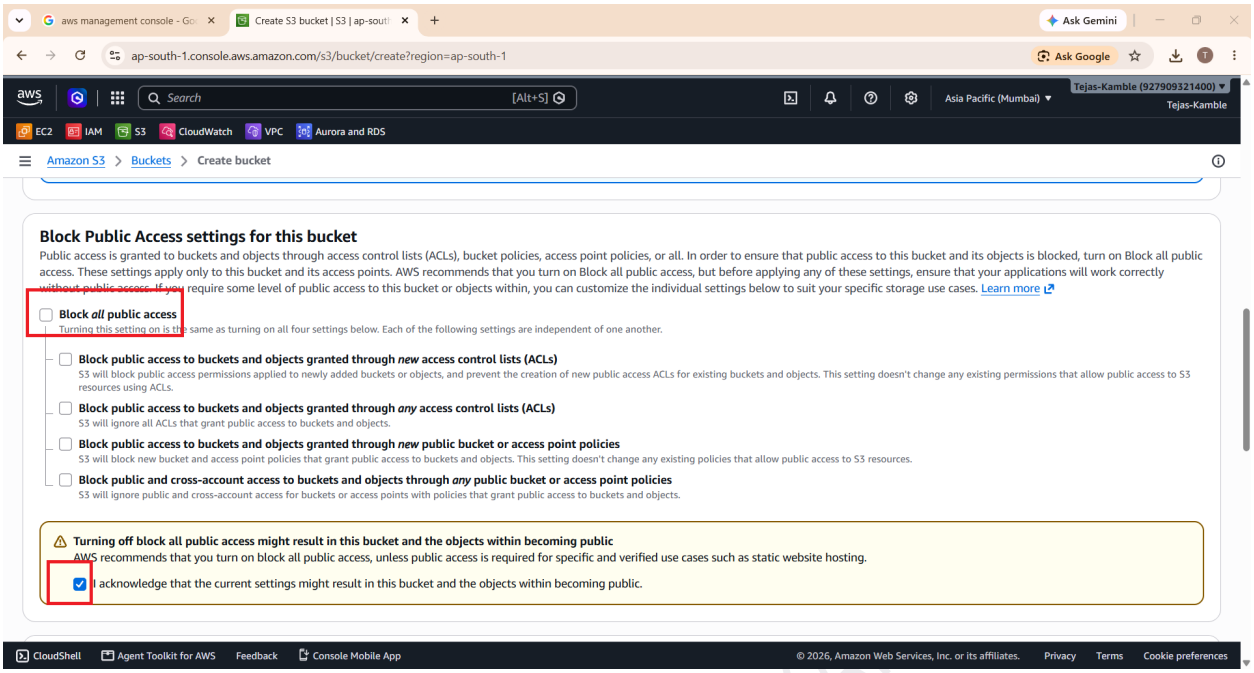


Figure 5. Public-access blocking options in the bucket creation workflow.

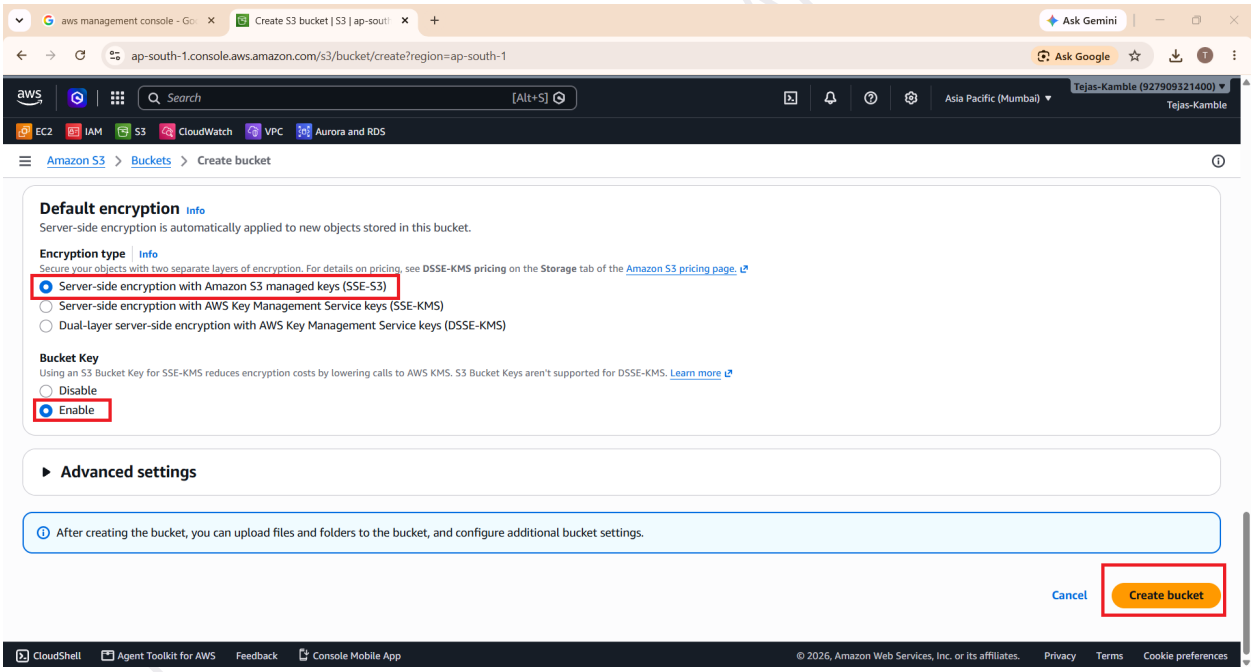


Figure 6. Default encryption configuration: SSE-S3 selected, S3 Bucket Key enabled, and Create bucket action.

4.2 Upload the website entry file

Screenshot 7 shows the newly created bucket's Objects view. Screenshot 8 shows the upload dialog. Screenshot 9 is the local file picker with `index.html` selected. Screenshots 10–11 show the remaining upload configuration, including a public-read access choice and an AWS warning about public access. Screenshot 12 shows the successful upload result and the object listing.

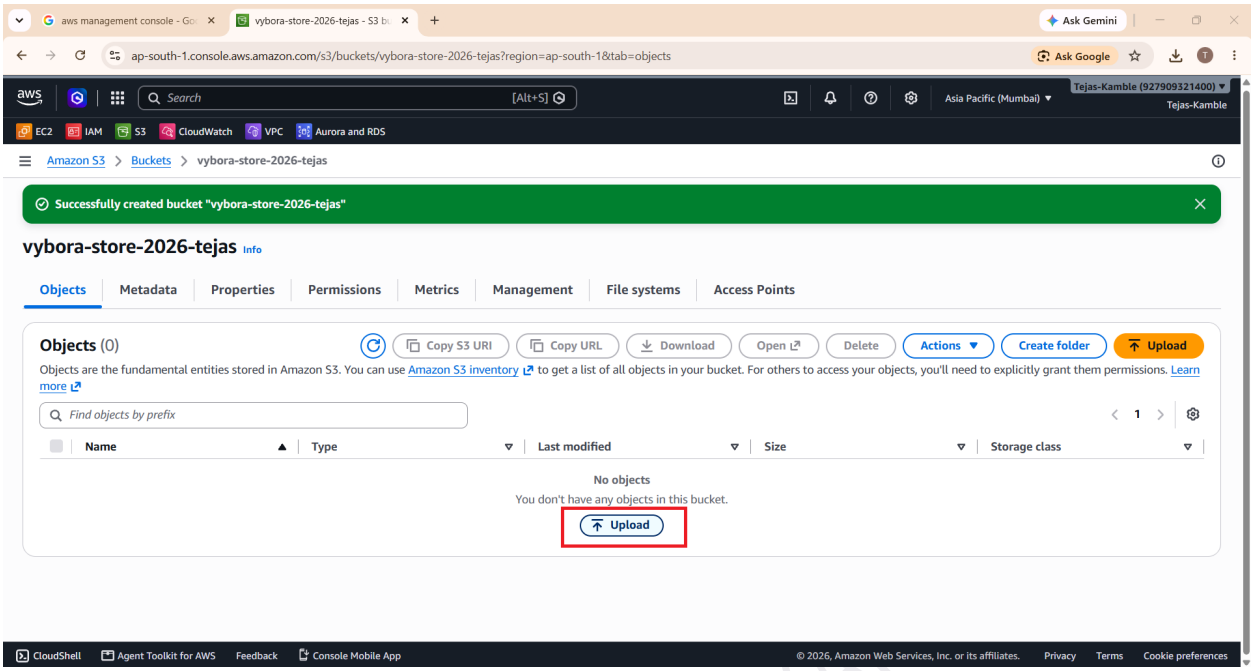


Figure 7. New bucket Objects view before the website object is uploaded.

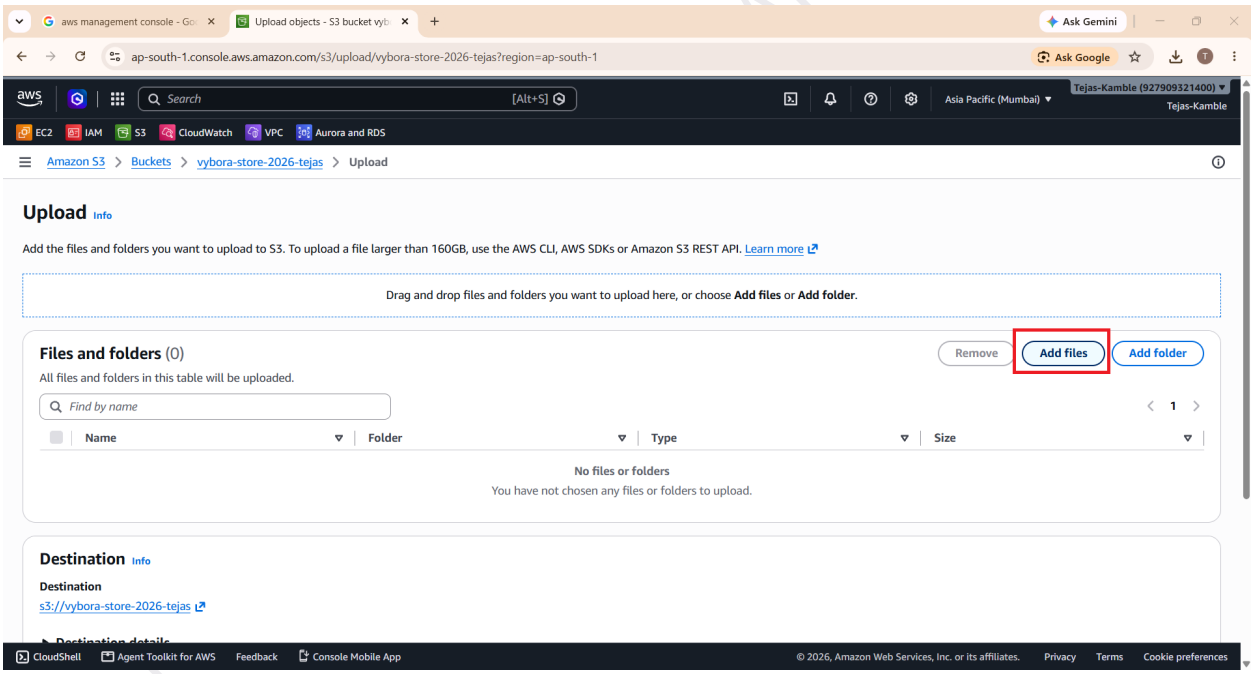


Figure 8. S3 upload form with Add files selected.

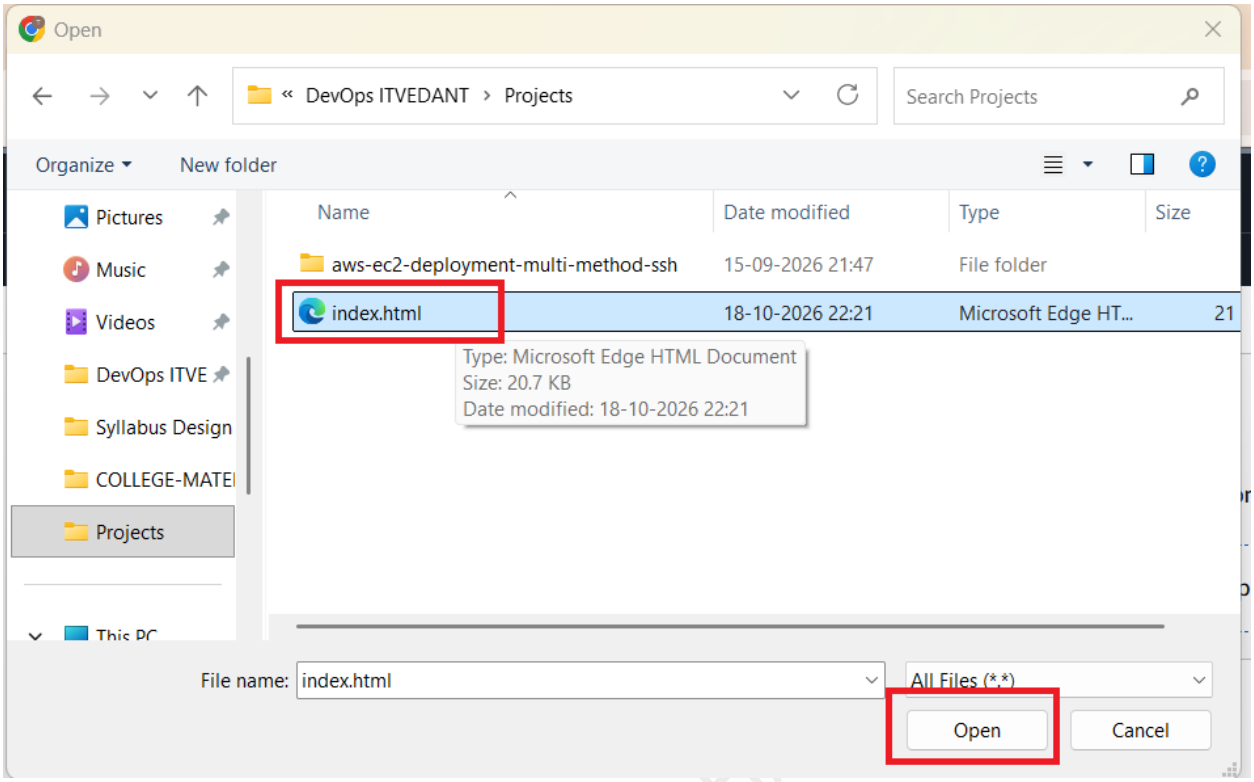


Figure 9. Local file selection: index.html.

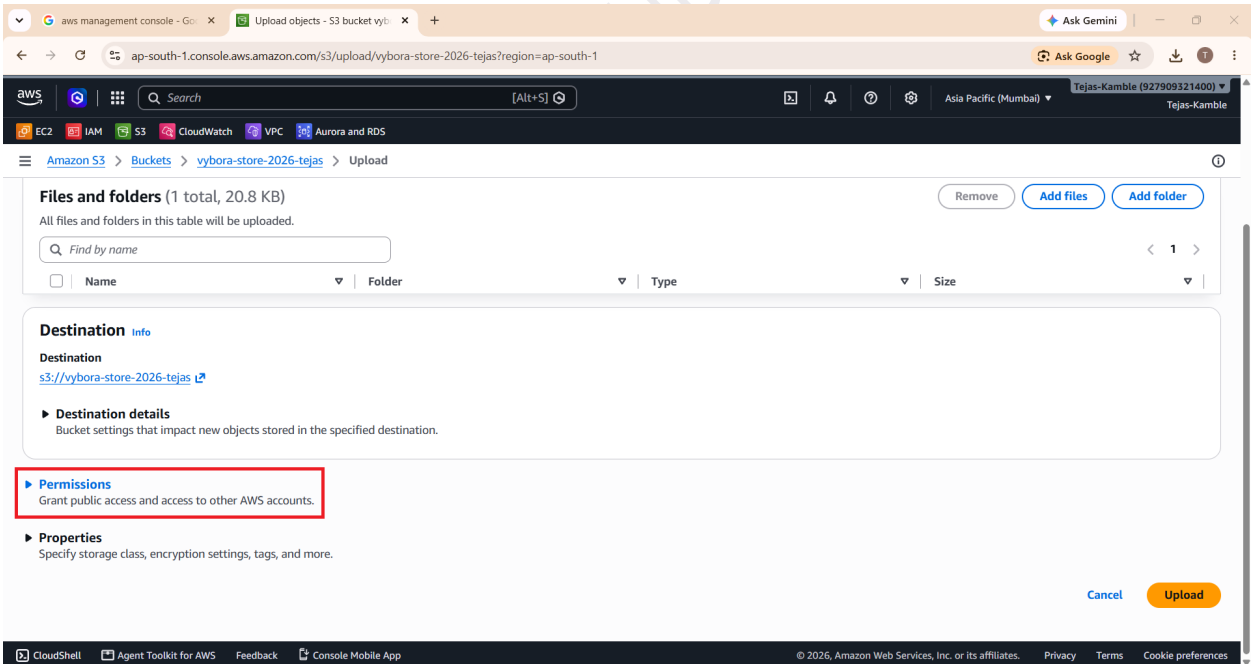


Figure 10. Upload settings and destination options.

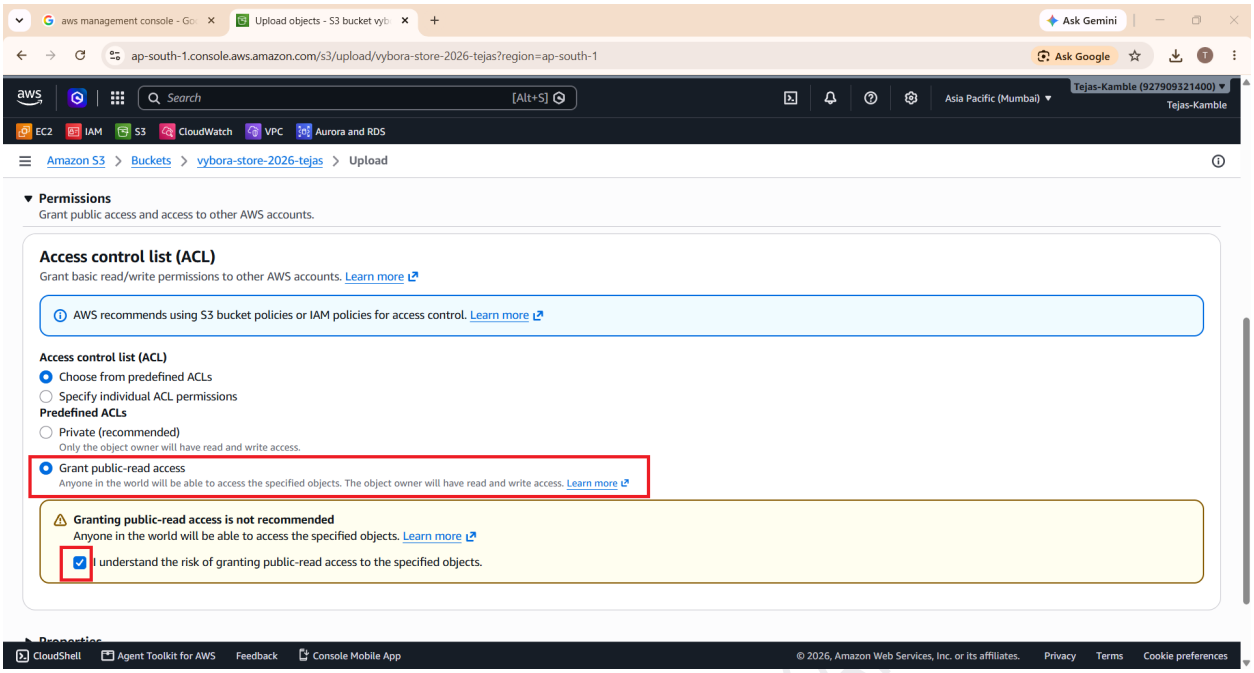


Figure 11. Public-read access option and associated warning in the upload workflow.

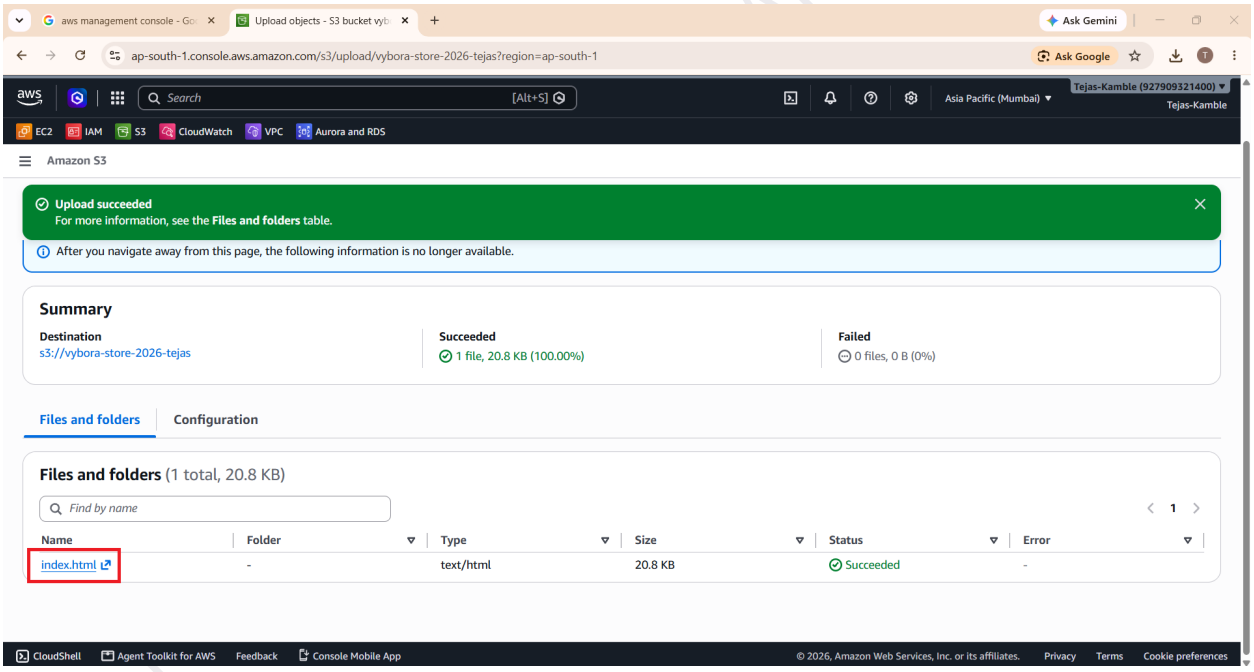


Figure 12. Upload success confirmation and uploaded object listing.

4.3 Obtain the website URL and verify in a browser

Screenshot 13 shows the uploaded object's details and a website URL reference. Screenshots 14–16 show the VYBORA storefront rendered in a browser: the hero section, category discovery area, and product cards. This establishes that the browser could render the website from the shown S3 website endpoint at the time of capture. It does not independently test every interaction, device size, or availability over time.

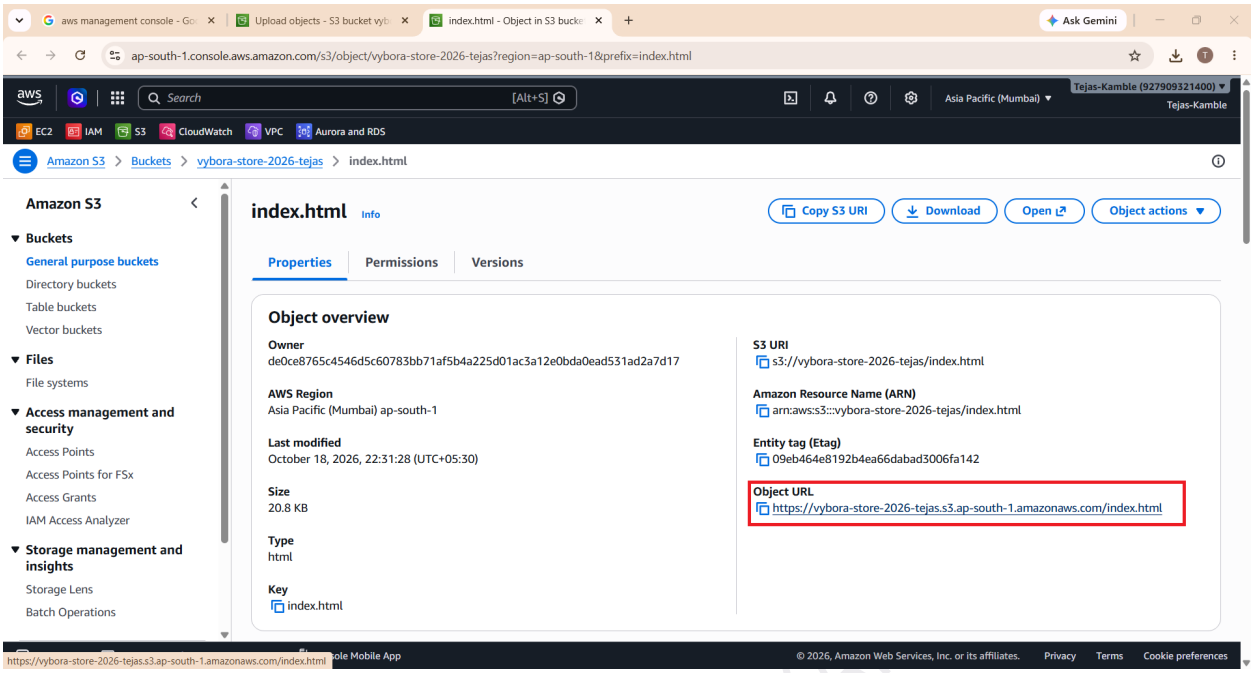


Figure 13. Uploaded object details and S3 website URL reference.

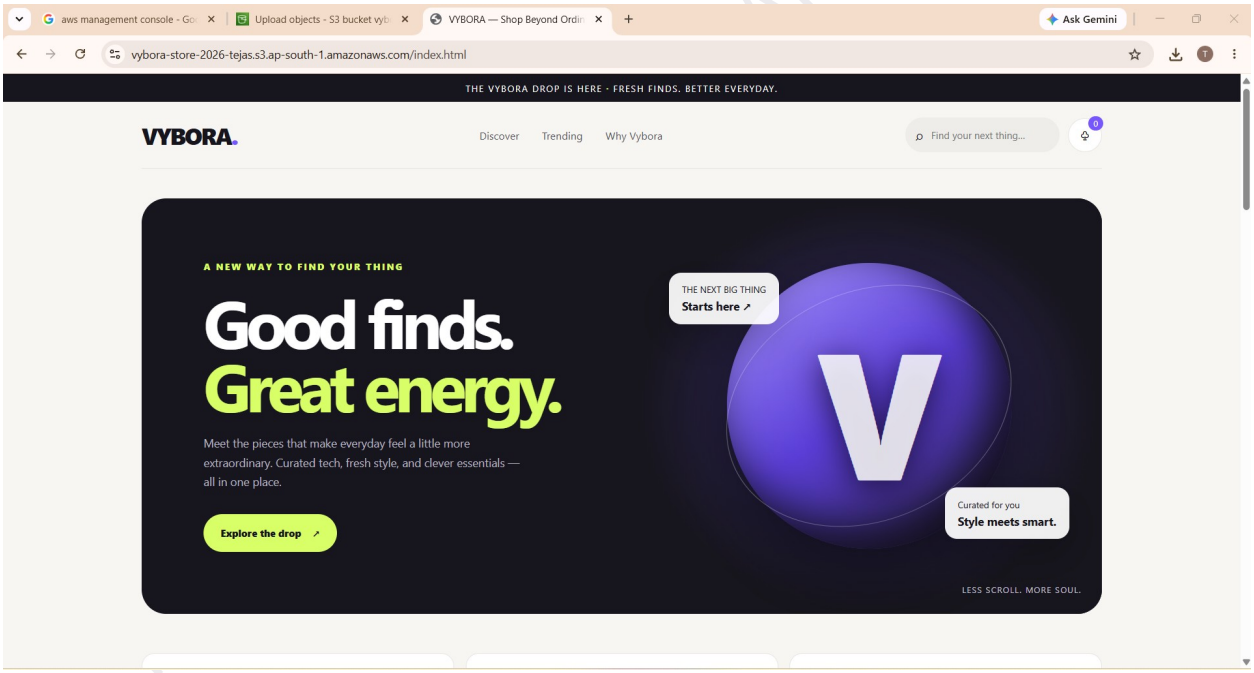


Figure 14. Browser rendering: storefront navigation and hero section.

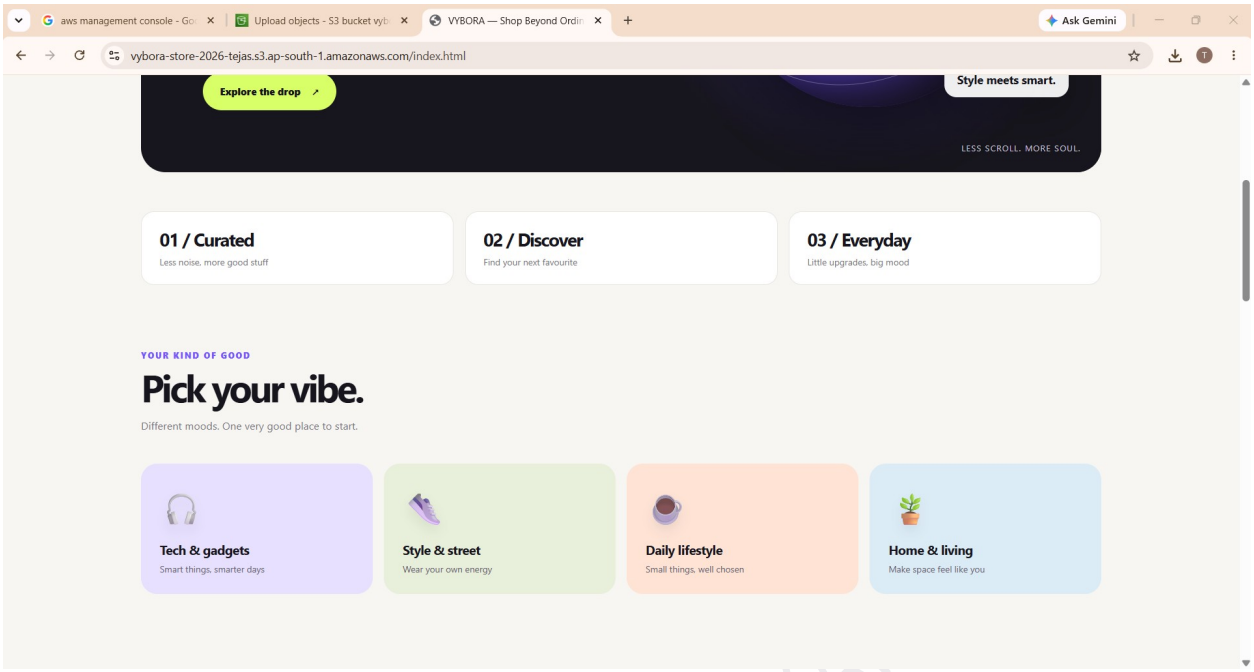


Figure 15. Browser rendering: store highlights and category tiles.

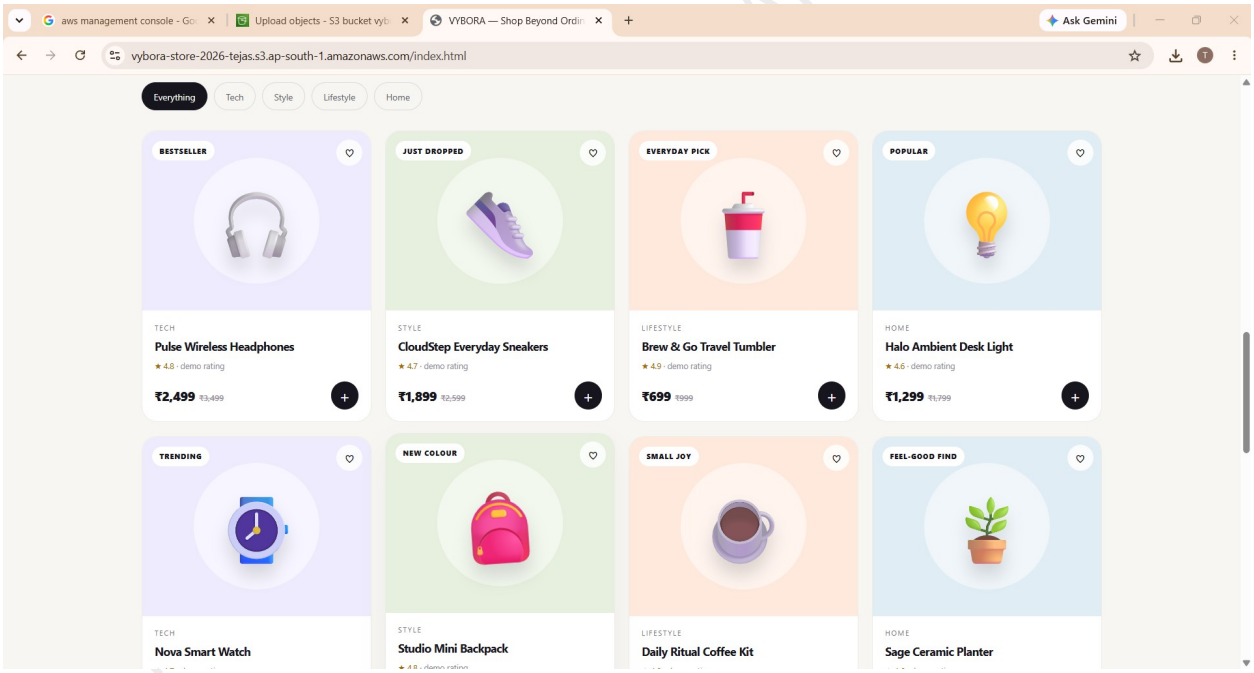


Figure 16. Browser rendering: product cards and storefront catalogue.

5. Testing and results

Evidence-based checks

- Deployment workflow: screenshots show bucket creation/configuration, object upload, and a successful upload message.

- Endpoint rendering: screenshots 14–16 show the storefront loading in a browser; screenshot 13 provides the endpoint context.
- Source inspection: the HTML includes page layout and client-side JavaScript for product filtering, search, demo cart count, and favourites.
- Responsive design evidence: source CSS contains breakpoints for narrower screens and a reduced-motion preference. The supplied screenshots do not constitute a mobile-device test.

Observed result

The supplied evidence supports the conclusion that the static VYBORA storefront was uploaded to an S3 bucket and rendered from an S3 website endpoint in a browser. The source supports a product-discovery demonstration with client-side interactions. No load, security, accessibility, cross-browser, or end-to-end purchase test results were included, so none are claimed here.

6. Security, limitations, and operational considerations

Security

The setup evidence includes public-read access selection and an AWS warning about public access. A public static website must serve its content to anonymous visitors; accordingly, bucket/object exposure should be limited to the intended website assets. Do not place credentials, private files, customer information, or other sensitive material in the public bucket. Review bucket policy, ACL/object ownership, and Block Public Access settings together, and grant only the access required for the chosen hosting architecture. The screenshots do not provide enough detail to audit the final effective permissions.

An S3 website endpoint is distinct from a TLS-enabled custom-domain setup. The archive does not show CloudFront or another HTTPS layer, DNS configuration, certificate setup, access logging, or security monitoring. These should be considered before presenting the site as production-ready.

Limitations

- Static single-page artifact: the provided project has no server-side application or API.
- Cart is demonstrative: the count is held in page memory and the cart message explicitly says checkout is not enabled.
- No payment, order, authentication, inventory, or customer-data workflow is evidenced.
- Product catalogue and ratings are client-side sample content, not verified live data.
- No build/deployment scripts, infrastructure-as-code, automated tests, or reproducible CI/CD are included in the supplied archive.
- The screenshots document a point-in-time manual workflow; they do not establish present-day availability or configuration state.

PROJECT BY TEJAS SUDHAKAR KAMBLE

7. Future improvements

- Add CloudFront in front of S3 for HTTPS delivery, custom-domain support, caching, and a controlled public access model.
- Automate deployment with a documented pipeline or infrastructure-as-code, with least-privilege deployment permissions and repeatable environment configuration.
- Connect a secure backend/API for catalogue, inventory, cart, identity, and order operations; integrate payments only through a compliant provider flow.
- Introduce automated checks for links, HTML semantics, accessibility, responsiveness, security headers, and browser behavior.
- Add operational readiness: logs/metrics, alerting, backup/versioning strategy, and documented rollback procedures.
- Document the exact S3 bucket policy, region, endpoint, and access model after verifying them in the AWS account; avoid publishing secrets or account-specific identifiers.

8. Conclusion

VYBORA demonstrates the core workflow for publishing a static storefront through Amazon S3: prepare a single-file website, configure a bucket for website hosting, upload the entry file, and verify the rendered page through the website endpoint. The supplied screenshots make the implementation sequence auditable, while source inspection clarifies that the shopping features are front-end demonstrations rather than a transactional commerce service. With HTTPS delivery, controlled access, automated deployment, and a secure commerce backend, the concept could be extended toward a production architecture.

PORTFOLIO SUMMARY Built a responsive static storefront concept and documented its manual deployment to Amazon S3 website hosting, with screenshot evidence from bucket setup through browser verification. The project demonstrates static web delivery—not payment processing or a production e-commerce backend.

Evidence index

All screenshots below are reproduced in the implementation section in their original numerical sequence. The source file reviewed was `Website-Landing-Page/index.html` from the supplied ZIP archive. The documentation intentionally avoids recording account identifiers or claiming configurations not verifiable from the evidence.

PROJECT BY TEJAS SUDHAKAR KAMBLE